

EX NO:01 Image Handling and Pixel Transformations Using OpenCV

Name:DEVADHAARINI.R

Register No:212225040061.

Step1: Load an image from your local directory and display it.

```
In [3]: import cv2
import matplotlib.pyplot as plt
```

```
In [4]: # Read the image using OpenCV
img = cv2.imread('2.jpg', cv2.IMREAD_COLOR)
```

```
In [5]: # Convert BGR (OpenCV's default) to RGB (Matplotlib's expected color order)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```
In [6]: # Display the image using Matplotlib
plt.imshow(img_rgb, cmap='viridis') # You can change 'viridis' to another cmap
plt.title("Original Image")
plt.axis('off') # Removes axis ticks and labels
plt.show()
```

Original Image



Step2: o Draw a line from the top-left to the bottom-right of the image. o Draw a circle at the center of the image. o Draw a rectangle around a specific region of interest in the image. o Add the text "OpenCV Drawing" at the top-left corner of the image. Draw a line from the top-left to the bottom-right of the image

```
In [7]: # Load the image
image = cv2.imread('2.jpg')
```

```
In [8]: # Convert BGR (OpenCV's default) to RGB (Matplotlib's expected color order)
img_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

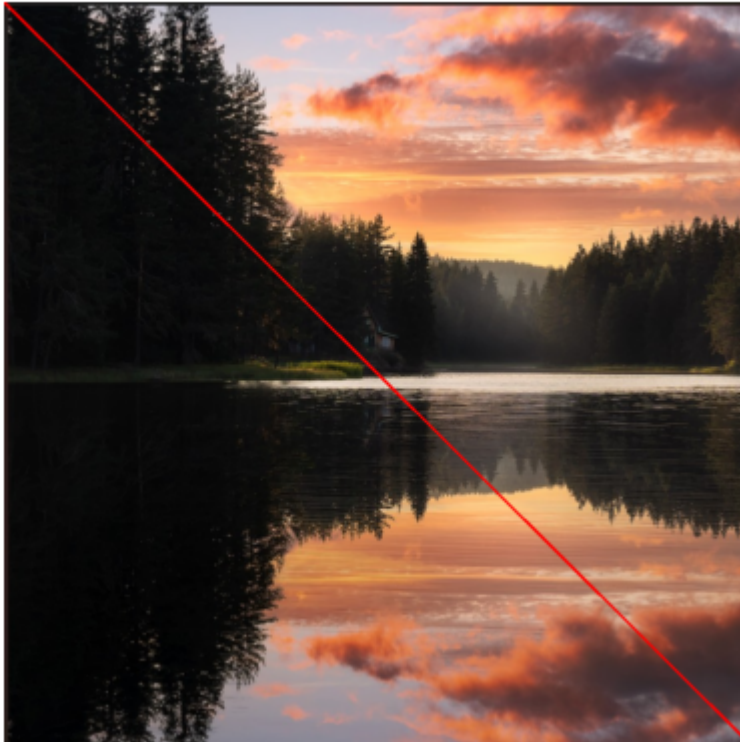
```
In [9]: img_rgb.shape
```

```
Out[9]: (902, 906, 3)
```

```
In [10]: # Draw a line from top-left to bottom-right  
line_img = cv2.line(img_rgb, (0, 0), (906, 902), (255, 0, 0), 2) # cv2.Line(image,
```

```
In [11]: plt.imshow(line_img, cmap='viridis')  
plt.title("Image with Line")  
plt.axis('off')  
plt.show()
```

Image with Line



Draw a circle at the center of the image.

```
In [12]: # Load the image  
image = cv2.imread('2.jpg')  
  
# Convert BGR (OpenCV's default) to RGB (Matplotlib's expected color order)  
img_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

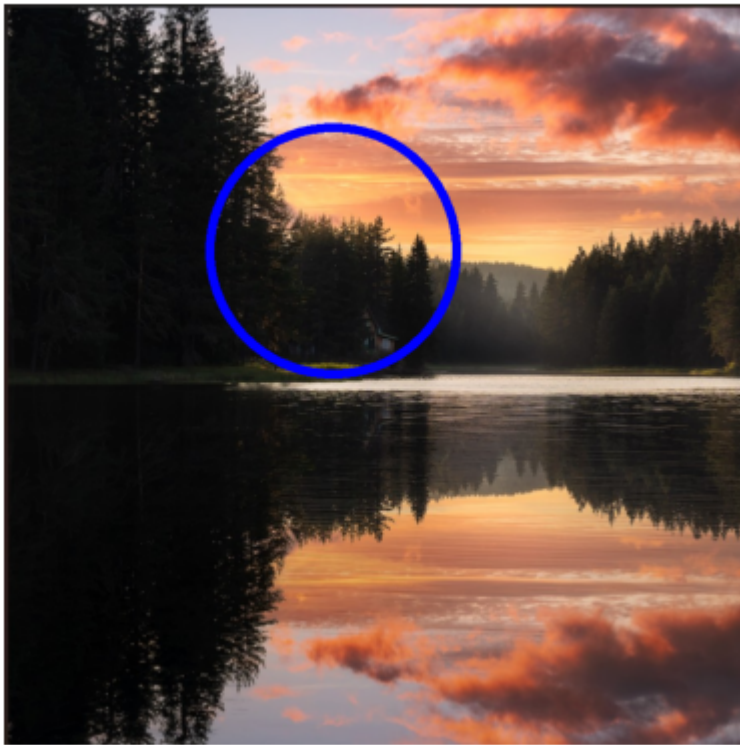
```
In [13]: img_rgb.shape
```

```
Out[13]: (902, 906, 3)
```

```
In [14]: circle_img = cv2.circle(img_rgb,(400,300),150,(0,0,255),10) # cv2.circle(image,
```

```
In [15]: plt.imshow(circle_img, cmap='viridis')  
plt.title("Image with Circle")  
plt.axis('off')  
plt.show()
```

Image with Circle



Draw a rectangle around the whole image

```
In [16]: # Load the image
image = cv2.imread('2.jpg')

# Convert BGR (OpenCV's default) to RGB (Matplotlib's expected color order)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```
In [17]: img.shape
```

```
Out[17]: (902, 906, 3)
```

```
In [18]: # Draw a rectangle around the whole image
rectangle_img = cv2.rectangle(img_rgb, (0, 0), (906, 902), (0, 255, 0), 10) # c
```

```
In [19]: plt.imshow(rectangle_img, cmap='viridis')
plt.title("Image with Rectangle")
plt.axis('off')
plt.show()
```

Image with Rectangle



Add the text "OpenCV Drawing" at the top-left corner of the image.

```
In [20]: # Load the image
image = cv2.imread('2.jpg')

# Convert BGR (OpenCV's default) to RGB (Matplotlib's expected color order)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```
In [21]: # Add text to the image
text_img = cv2.putText(img_rgb, "DEVADHAARINI", (10, 30), cv2.FONT_HERSHEY_SIMPL
```

```
In [22]: plt.imshow(text_img, cmap='viridis')
plt.title("Image with Text")
plt.axis('off')
plt.show()
```

Image with Text



Step3: o Convert the image from RGB to HSV and display it. o Convert the image from RGB to GRAY and display it. o Convert the image from RGB to YCrCb and display it. o Convert the HSV image back to RGB and display it.

```
In [21]: # Load the image
image = cv2.imread('2.jpg')
```

```
In [22]: image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

```
In [23]: # Original RGB Image
plt.imshow(image_rgb)
plt.title("Original RGB Image")
plt.axis("off")
```

```
Out[23]: (np.float64(-0.5), np.float64(905.5), np.float64(901.5), np.float64(-0.5))
```

Original RGB Image

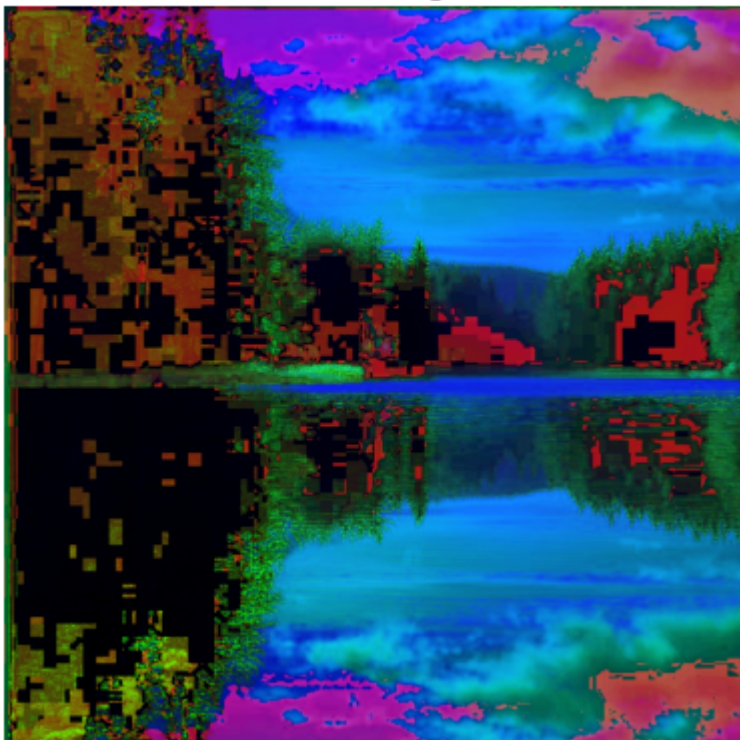


```
In [24]: # Convert RGB to HSV  
image_hsv = cv2.cvtColor(image_rgb, cv2.COLOR_RGB2HSV)
```

```
In [25]: # HSV Image  
plt.imshow(image_hsv)  
plt.title("HSV Image")  
plt.axis("off")
```

```
Out[25]: (np.float64(-0.5), np.float64(905.5), np.float64(901.5), np.float64(-0.5))
```

HSV Image



```
In [26]: # Convert RGB to GRAY
image_gray = cv2.cvtColor(image_rgb, cv2.COLOR_RGB2GRAY)
```

```
In [27]: # Grayscale Image
plt.imshow(image_gray, cmap='gray')
plt.title("Grayscale Image")
plt.axis("off")
```

```
Out[27]: (np.float64(-0.5), np.float64(905.5), np.float64(901.5), np.float64(-0.5))
```

Grayscale Image



```
In [28]: # Convert RGB to YCrCb
image_ycrb = cv2.cvtColor(image_rgb, cv2.COLOR_RGB2YCrCb)
```

```
In [29]: # YCrCb Image
plt.imshow(image_ycrb)
plt.title("YCrCb Image")
plt.axis("off")
```

```
Out[29]: (np.float64(-0.5), np.float64(905.5), np.float64(901.5), np.float64(-0.5))
```

YCrCb Image



```
In [30]: # Convert HSV back to RGB  
image_hsv_to_rgb = cv2.cvtColor(image_hsv, cv2.COLOR_HSV2RGB)
```

```
In [31]: plt.imshow(image_hsv_to_rgb)  
plt.title("HSV to RGB Image")  
plt.axis("off")
```

```
Out[31]: (np.float64(-0.5), np.float64(905.5), np.float64(901.5), np.float64(-0.5))
```

HSV to RGB Image



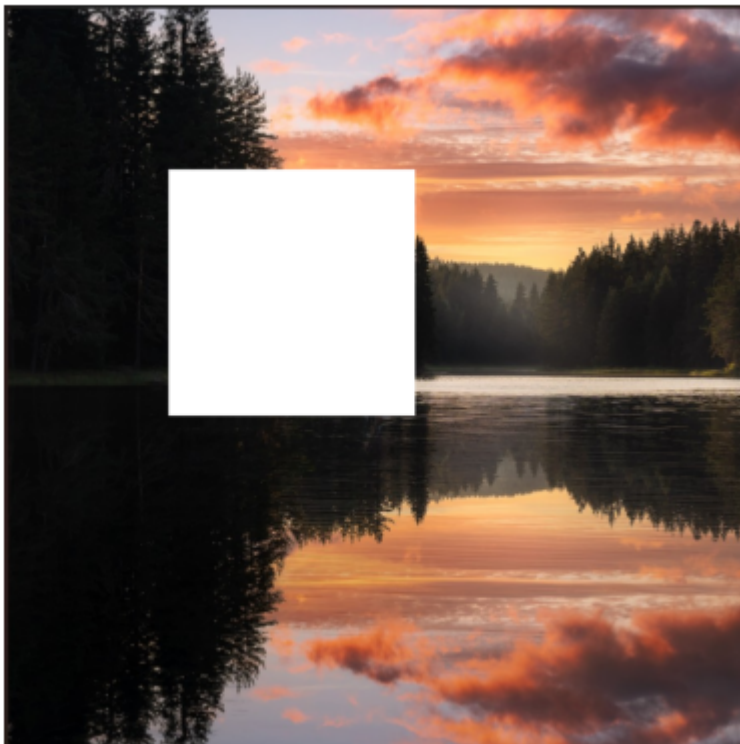
Step4: o Access and print the value of the pixel at coordinates (100, 100). o Modify the color of the pixel at (200, 200) to white.

```
In [32]: # Modify a block of pixels (300x300) to white, starting from (200, 200)
image[200:500, 200:500] = [255, 255, 255] # Rows: 200-499, Columns: 200-499
```

```
In [33]: # Convert BGR to RGB for displaying with Matplotlib
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

```
In [34]: # Display the modified image
plt.imshow(image_rgb)
plt.title("Image with 300x300 White Block")
plt.axis("off")
plt.show()
```

Image with 300x300 White Block



Step5: o Resize the original image to half its size and display it.

```
In [35]: # Load the image
image = cv2.imread('2.jpg')
```

```
In [36]: image.shape
```

```
Out[36]: (902, 906, 3)
```

```
In [37]: # Resize the image to half its size
resized_image = cv2.resize(image, (906 // 2, 902 // 2)) # (new_width, new_height)
```

```
In [38]: # Convert BGR to RGB for displaying with Matplotlib
resized_image_rgb = cv2.cvtColor(resized_image, cv2.COLOR_BGR2RGB)
```

```
In [39]: resized_image_rgb.shape
```

```
Out[39]: (451, 453, 3)
```

```
In [40]: # Display the resized image
plt.imshow(resized_image_rgb)
plt.title("Resized Image (Half Size)")
plt.axis("off")
plt.show()
```

Resized Image (Half Size)



Step6: o Crop a region of interest (ROI) from the image (e.g., a 100x100 pixel area starting at (50, 50)) and display it.

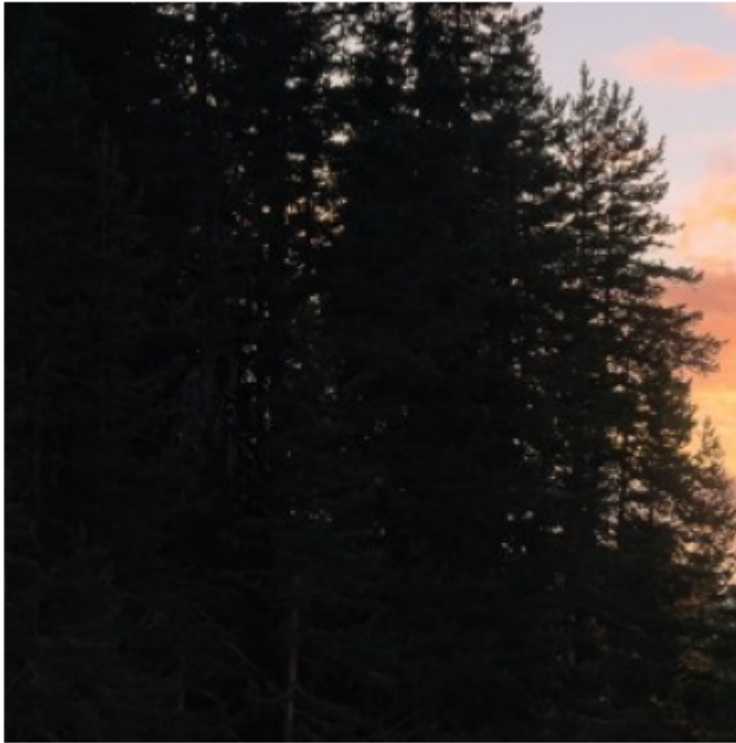
```
In [41]: # Load the image
image = cv2.imread('2.jpg')
```

```
In [42]: # Crop a 300x300 region starting from (50, 50)
roi = image[50:350, 50:350] # Rows: 50-349, Columns: 50-349
```

```
In [43]: # Convert BGR to RGB for displaying with Matplotlib
roi_rgb = cv2.cvtColor(roi, cv2.COLOR_BGR2RGB)
```

```
In [44]: # Display the cropped region (ROI)
plt.imshow(roi_rgb)
plt.title("Cropped Region of Interest (ROI)")
plt.axis("off")
plt.show()
```

Cropped Region of Interest (ROI)



Step7: o Flip the original image horizontally and display it. o Flip the original image vertically and display it.

```
In [45]: # Load the image
image = cv2.imread('2.jpg')
```

```
In [46]: # Flip the image horizontally (left-right)
flipped_horizontally = cv2.flip(image, 1)
```

```
In [47]: # Convert BGR to RGB for displaying with Matplotlib
flipped_horizontally_rgb = cv2.cvtColor(flipped_horizontally, cv2.COLOR_BGR2RGB)
```

```
In [48]: # Horizontal flip
plt.imshow(flipped_horizontally_rgb)
plt.title("Flipped Horizontally")
plt.axis("off")
```

```
Out[48]: (np.float64(-0.5), np.float64(905.5), np.float64(901.5), np.float64(-0.5))
```

Flipped Horizontally



```
In [49]: # Flip the image vertically (up-down)
         flipped_vertically = cv2.flip(image, 0)
```

```
In [50]: # Convert BGR to RGB for displaying with Matplotlib
         flipped_vertically_rgb = cv2.cvtColor(flipped_vertically, cv2.COLOR_BGR2RGB)
```

```
In [51]: # Vertical flip
         plt.imshow(flipped_vertically_rgb)
         plt.title("Flipped Vertically")
         plt.axis("off")
```

```
Out[51]: (np.float64(-0.5), np.float64(905.5), np.float64(901.5), np.float64(-0.5))
```

Flipped Vertically



Step8: o Save the final modified image to your local directory.